Social Media Profile Manager – Django Project

Section A: Conceptual Understanding

1. Django Request-Response Cycle vs Standard Python Script

Django Request-Response Cycle

Django follows a web-based request-response architecture.

Flow:

1. User sends request from browser.
2. Request goes to Django server.
3. URL dispatcher checks urls.py.
4. Matching view function is executed.
5. View communicates with models/database.
6. Data is passed to template.
7. HTML response is returned to browser.

Standard Python Script Execution

A normal Python script runs line-by-line from top to bottom and stops after execution.

Difference

Django	Standard Python Script
Works with web requests	Runs directly in terminal
Uses URLs, Views, Templates	Uses simple sequential code
Handles multiple users	Usually single execution
Connected with database	May or may not use database
Dynamic web output	Static console output

---

2. Why Django Model Fields are More Robust

Django model fields like CharField and IntegerField provide strong validation and database structure.

Benefits:

- Data type checking
- Automatic validation
- Prevents invalid data
- Database compatibility
- Easier querying
- Better security

Example

```
username = models.CharField(max_length=100)
age = models.IntegerField()
```

If a string is entered in age field, Django prevents invalid data.

Python dynamic typing allows:

```
age = "hello"
```

which can create errors later.

---

3. Django Forms Validation

Django Forms automatically validate user inputs.

Example:

```
class UserProfileForm(forms.ModelForm):

    def clean_age(self):
        age = self.cleaned_data['age']

        if age < 13:
            raise forms.ValidationError("Age must be above 13")

        return age
```

Validation Features

- Required fields checking
- Data type validation
- Custom validation methods

- Error messages
- Secure input handling

---

4. Conditional Logic in Django Templates

Django Template Language (DTL) supports conditional statements.

Example:

```
{% if profile.is_public %}
    <p>Public Account</p>
{% else %}
    <p>Private Account</p>
{% endif %}
```

Use Cases

- Toggle visibility
- Show/hide content
- Role-based display
- Dynamic user interface

---

5. Python List vs Django QuerySet

Python List

```
users = ["Jay", "Rahul", "Aman"]
```

- Stored in memory
- Static data
- No database connection

Django QuerySet

```
profiles = UserProfile.objects.all()
```

- Retrieves data from database
- Lazy loading
- Supports filtering and querying
- Dynamic and scalable

Difference

Python List	Django QuerySet
In-memory data	Database data
Fixed values	Dynamic values
No ORM support	ORM support
Fast for small data	Better for large data

---

6. Why Django ORM is Preferred

Django ORM allows developers to work with database using Python code instead of raw SQL.

Advantages

- Easy database operations
- Secure against SQL injection
- Better readability
- Database portability
- Faster development
- Relationship handling

Example

```
UserProfile.objects.create(username="Jay", age=20)
```

Compared to dictionaries:

```
user = {
    "username": "Jay",
    "age": 20
}
```

Dictionary data disappears after program closes, but ORM stores data permanently in database.

Section B: Practical Tasks

1. UserProfile Model

models.py

```
from django.db import models

class UserProfile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    is_public = models.BooleanField(default=True)

    def __str__(self):
        return self.username
```

---

2. ModelForm with Validation

forms.py

```
from django import forms
from .models import UserProfile

class UserProfileForm(forms.ModelForm):

    class Meta:
        model = UserProfile
        fields = ['username', 'age', 'is_public']

    def clean_age(self):
        age = self.cleaned_data['age']

        if age < 13:
            raise forms.ValidationError("User must be older than 13")

        return age
```

---

3. Django Views for Persistence

views.py

```
from django.shortcuts import render, redirect
from .forms import UserProfileForm
from .models import UserProfile


def create_profile(request):

    if request.method == 'POST':
        form = UserProfileForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect('profile_list')

    else:
        form = UserProfileForm()

    return render(request, 'create_profile.html', {'form': form})
```

---

4. Render Profiles with DTL

views.py

```
from django.shortcuts import render
from .models import UserProfile


def profile_list(request):
    profiles = UserProfile.objects.all()

    return render(request, 'profile_list.html', {
        'profiles': profiles
    })
```

profile_list.html

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title>Profile List</title>
</head>
<body>

<h1>User Profiles</h1>

<ul>
    {% for profile in profiles %}
        <li>
            {{ profile.username }} - {{ profile.age }}
        </li>
    {% endfor %}
</ul>

</body>
</html>
```

Section C: Mini Project

Social Profile Web App

Objective

Create a Django-based web application for managing social media profiles with database storage and CSV export functionality.

Project Structure

```
social_project/
│
├── manage.py
├── social_project/
│   ├── settings.py
│   ├── urls.py
│
├── profiles/
│   ├── models.py
│   ├── views.py
│   ├── forms.py
```

```
│   ├── urls.py
│   ├── templates/
│   │   ├── create_profile.html
│   │   └── profile_list.html
```

Step 1: Create/Edit using Django Forms

create_profile.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Create Profile</title>
</head>
<body>

<h1>Create User Profile</h1>

<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Save Profile</button>
</form>

</body>
</html>
```

Step 2: Database Integration

Install MySQL Driver

```
pip install mysqlclient
```

settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
```

```
        'NAME': 'socialdb',
        'USER': 'root',
        'PASSWORD': '',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 3: Export Data to CSV

views.py

```
import csv
from django.http import HttpResponse
from .models import UserProfile


def export_profiles(request):

    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = 'attachment; filename="profiles.csv"'

    writer = csv.writer(response)

    writer.writerow(['Username', 'Age', 'Public'])

    profiles = UserProfile.objects.all()

    for profile in profiles:
        writer.writerow([
            profile.username,
            profile.age,
            profile.is_public
        ])

    return response
```

Context Manager Example

```
with open('profiles.csv', 'w') as file:
    file.write('Data Saved')
```

Step 4: URL Routing

profiles/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.profile_list, name='profile_list'),
    path('create/', views.create_profile, name='create_profile'),
    path('export/', views.export_profiles, name='export_profiles'),
]
```

Main urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('profiles.urls')),
]
```

Output Features

Commands to Run Project

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Full Django Project Coding

Create Project

```
django-admin startproject social_project
cd social_project
python manage.py startapp profiles
```

---

settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'profiles',
]
```

---

models.py

```
from django.db import models

class UserProfile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    is_public = models.BooleanField(default=True)

    def __str__(self):
        return self.username
```

---

forms.py

```
from django import forms
from .models import UserProfile
```

```
class UserProfileForm(forms.ModelForm):

    class Meta:
        model = UserProfile
        fields = ['username', 'age', 'is_public']

    def clean_age(self):
        age = self.cleaned_data['age']

        if age < 13:
            raise forms.ValidationError('Age must be above 13')

        return age
```

---

views.py

```
import csv
from django.shortcuts import render, redirect
from django.http import HttpResponse
from .forms import UserProfileForm
from .models import UserProfile


def profile_list(request):

    profiles = UserProfile.objects.all()

    return render(request, 'profile_list.html', {
        'profiles': profiles
    })


def create_profile(request):

    if request.method == 'POST':
        form = UserProfileForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect('profile_list')

    else:
        form = UserProfileForm()
```

```
    return render(request, 'create_profile.html', {
        'form': form
    })


def export_profiles(request):

    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = 'attachment; filename="profiles.csv"'

    writer = csv.writer(response)

    writer.writerow([
        'Username',
        'Age',
        'Public Status'
    ])

    profiles = UserProfile.objects.all()

    for profile in profiles:
        writer.writerow([
            profile.username,
            profile.age,
            profile.is_public
        ])

    return response
```

---

profiles/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.profile_list, name='profile_list'),
    path('create/', views.create_profile, name='create_profile'),
    path('export/', views.export_profiles, name='export_profiles'),
]
```

---

main urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('profiles.urls')),
]
```

---

create_profile.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Create Profile</title>
</head>
<body>

<h1>Create Profile</h1>

<form method="POST">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Save</button>
</form>

<a href="/">View Profiles</a>

</body>
</html>
```

---

profile_list.html

```
<html>
<head>
    <title>Profile List</title>
</head>
```

```
<body>

<h1>All Profiles</h1>

<a href="/create/">Create Profile</a>
<a href="/export/">Export CSV</a>

<hr>

<ul>
    {% for profile in profiles %}

        <li>
            {{ profile.username }} - {{ profile.age }}

            {% if profile.is_public %}
                (Public)
            {% else %}
                (Private)
            {% endif %}
        </li>

    {% endfor %}
</ul>

</body>
</html>
```